

```
/*
=====
=====

    sk-progress.js – розрахунок Health та
    XP з прогресу + збереження у Firebase
    Підключення (звичайний скрипт, не
    модуль):
        <script src="sk-progress.js">
    </script>
    Працює і без Firebase (тоді просто
    рахує значення з localStorage).

    ЛОГІКА (1 клас):
        • HEALTH = 50 базових + до +50,
        розподілених ПОРІВНУ між завданнями
        з рідної мови (українська). Кожне
        завдання дає свою частку
        пропорційно до того, наскільки воно
        виконане ("по мірі виконання").
        Зараз 5 слотів → по 10 НР. Усі
        виконані → 100. Стеля – 100.
        • XP = +1 за кожен виконаний вправу у
        КВЕСТАХ 1 класу (всі предмети).
        Тренування у XP НЕ входять.
        • Обидва показники зберігаються у
        Firebase: SK.saveHeroStats({health, xp}).

    Щоб додати новий квест з української –
    допиши запис у UKR_TASKS
```

(заміни зарезервований слот) і, за потреби, ключ-лічильник у QUEST_XP_KEYS.

```
=====
===== */
(function () {
  'use strict';

  var BASE_HEALTH = 50;          //
початкове здоров'я
  var HEALTH_FROM_TASKS = 50; // скільки
сумарно дають усі завдання рідної мови
  var MAX_HEALTH = 100;         //
довідкове значення (ПОКИ не стеля –
здоров'я не обмежується)

  /* — ТОЧНІСТЬ (accuracy) – стат від
скілів МАТЕМАТИКИ. —
    Базове значення = 1 (як у
defaultHero). Кожен ПРОЙДЕНИЙ БЕЗ ПОМИЛОК
    рівень математичного скіла додає +1
до точності.
    Скіл записує множину пройдених
рівнів у свій localStorage-ключ
    (JSON-об'єкт {"1":true,...}); тут ми
лише рахуємо їх кількість.
    ПОКИ без стелі – точність
накопичується без обмеження.
    Поле `levels` – довідкове (скільки
рівнів у скілі), не обмежує суму.
    Щоб додати новий математичний скіл –
допиши сюди {key, levels}. */
```

```

var BASE_ACCURACY = 1;
var ACCURACY_SKILLS = [
    { key: 'sk_add10_levels_v1', levels:
10 }, // Додавання до 10
    { key: 'sk_sub10_levels_v1', levels:
10 } // Віднімання до 10
    // майбутні мат-скіли: {
key:'sk_add20_levels_v1', levels:10 },
    ...
];

/* — Завдання з РІДНОЇ МОВИ (дають
HEALTH). Кожне – рівна частка з +50. —
frac() повертає 0..1 – наскільки
завдання виконане. */
var UKR_TASKS = [
    {
        id: 'ukr_train_letters',
        label: 'Тренування: Букви Абетки',
        kind: 'training',
        frac: function () { return
setFrac('sk_train_ukrabc_learned_v1',
32); }
    },
    {
        id: 'ukr_quest_spelling',
        label: 'Квест: Правопис слів',
        kind: 'quest',
        frac: function () { return
mapFrac('sk_spell_done_v1', 10); }
    },
    // — місце для 3 майбутніх квестів з

```

української —

```
    { id: 'ukr_quest_slot2', label:
      'Квест з української (скопо)', kind:
      'quest', reserved: true, frac: function
      () { return 0; } },
    { id: 'ukr_quest_slot3', label:
      'Квест з української (скопо)', kind:
      'quest', reserved: true, frac: function
      () { return 0; } },
    { id: 'ukr_quest_slot4', label:
      'Квест з української (скопо)', kind:
      'quest', reserved: true, frac: function
      () { return 0; } }
  ];
```

```
/* — Лічильники ХР з РІДНОЇ МОВИ
(показуємо поряд зі здоров'ям). —
  Сюди входять і тренування, і квести
української. */
var UKR_XP_KEYS = [
  'sk_train_ukrabc_pts_v1', //
Тренування: Букви Абетки
  'sk_spell_pts_v1'         // Квест:
Правопис слів
  // майбутні квести з української
додавати сюди
];
```

```
/* — Лічильники виконаних вправ у
КВЕСТАХ (дають ХР, +1 за вправу). —
  Тренування СЮДИ не входять. Явні
ключі відомих квестів: */
```

```

var QUEST_XP_KEYS = [
    'sk_spell_pts_v1'    // Українська ·
    Правопис слів (+1 за правильну вправу)
    // майбутні квести додавати сюди,
    напр.: 'sk_palace_xp_v1', 'mathtest_xp',
    ...
];
/* Решту квестових '*хр*'-ключів
підбираємо автоматично за підрядками
предметів,
    щоб уже наявні квести враховувались
без ручного переліку.
    Тренування використовують '_pts_'
без 'хр' — тож сюди не потраплять. */
var QUEST_XP_PATTERNS = [
    'palace', 'rak', 'piven',
    'druzhba',           // українська-квести
    'math', 'add', 'sub', 'mult', 'num',
    'cifr', // математика
    'capital', 'geo', 'svit', 'world',
    'yads', 'dovkil', // світ
    'eng',
    'abc',               //
    англійська
    'logic', 'logika', 'pamyat',
    'memory'             // логіка/пам'ять
];

/* — допоміжні — */
function num(key) {
    var v =
    parseInt(localStorage.getItem(key) ||

```

```

'0', 10);
    return isNaN(v) ? 0 : v;
}
// частка за JSON-мапою виконаних
елементів {"1":true,...} → keys/total
function mapFrac(key, total) {
    try {
        var o =
JSON.parse(localStorage.getItem(key) ||
'{}');
        var n = (o && typeof o ===
'object') ? Object.keys(o).length : 0;
        return clamp01(n / total);
    } catch (e) { return 0; }
}
// частка за множиною вивчених
елементів (JSON-масив або мапа) →
size/total
function setFrac(key, total) {
    try {
        var a =
JSON.parse(localStorage.getItem(key) ||
'[]');
        var n = Array.isArray(a) ? a.length
: ((a && typeof a === 'object') ?
Object.keys(a).length : 0);
        return clamp01(n / total);
    } catch (e) { return 0; }
}
function clamp01(x) { return
Math.max(0, Math.min(1, x)); }

```

```

/* — HEALTH — */
// ПОКИ без стелі: прибрано верхнє
обмеження 100 (лишається лише підлога –
базове здоров'я).
function computeHealth() {
    var n = UKR_TASKS.length || 1;
    var share = HEALTH_FROM_TASKS / n;
// напр. 50/5 = 10
    var bonus = 0;
    for (var i = 0; i < UKR_TASKS.length;
i++) {
        bonus += share *
clamp01(UKR_TASKS[i].frac());
    }
    return Math.max(BASE_HEALTH,
Math.round(BASE_HEALTH + bonus));
}

// кількість елементів у множині
пройдених рівнів (JSON-об'єкт або масив)
function countSet(key) {
    try {
        var o =
JSON.parse(localStorage.getItem(key) ||
'{}');
        if (Array.isArray(o)) return
o.length;
        return (o && typeof o === 'object')
? Object.keys(o).length : 0;
    } catch (e) { return 0; }
}

```

```

/* — ТОЧНІСТЬ (accuracy) — */
// ПОКИ без стелі: точність просто
накопичується (base + усі пройдені
рівні).
function computeAccuracy() {
    var acc = BASE_ACCURACY;
    for (var i = 0; i <
ACCURACY_SKILLS.length; i++) {
        acc +=
countSet(ACCURACY_SKILLS[i].key);
    }
    return acc;
}

/* — ХР (виконані вправи квестів) —
*/
function computeXP() {
    var xp = 0, seen = {};
    for (var i = 0; i <
QUEST_XP_KEYS.length; i++) {
        var k = QUEST_XP_KEYS[i];
        xp += num(k); seen[k] = true;
    }
    try {
        for (var j = 0; j <
localStorage.length; j++) {
            var key = localStorage.key(j);
            if (!key || seen[key]) continue;
            var kl = key.toLowerCase();
            if (kl.indexOf('xp') === -1)
continue;
            for (var p = 0; p <

```



```

QUEST_XP_PATTERNS.length; p++) {
    if
    (kl.indexOf(QUEST_XP_PATTERNS[p]) !== -1)
    {
        var v =
    parseInt(localStorage.getItem(key) ||
    '0', 10);
        if (!isNaN(v)) xp += v;
        break;
    }
    }
    } catch (e) {}
    return xp;
}

/* — XP рідної мови (тренування +
квести української) для показу біля
здоров'я — */
function computeUkrXP() {
    var xp = 0;
    for (var i = 0; i <
    UKR_XP_KEYS.length; i++) xp +=
    num(UKR_XP_KEYS[i]);
    return xp;
}

/* — знімок усіх показників (для UI)
— */
function snapshot() {
    var tasks = [];
    for (var i = 0; i < UKR_TASKS.length;

```

```

i++) {
    var t = UKR_TASKS[i];
    tasks.push({ id: t.id, label:
t.label, kind: t.kind, reserved:
!!t.reserved, percent:
Math.round(clamp01(t.frac()) * 100) });
}
return {
    health: computeHealth(),
    maxHealth: MAX_HEALTH,
    accuracy: computeAccuracy(),
    xp: computeXP(),
    ukrXP: computeUkrXP(),
    healthPerTask: HEALTH_FROM_TASKS /
(UKR_TASKS.length || 1),
    ukrTasks: tasks
};
}

/* — збереження у Firebase (health +
xp) — */
function save() {
    var s = snapshot();
    try {
        if (window.SK && SK.ready && typeof
SK.ready.then === 'function') {
            SK.ready.then(function () {
                if (SK.currentUser &&
SK.currentUser() && SK.activeChildId) {
                    SK.saveHeroStats({ health:
s.health, accuracy: s.accuracy, xp: s.xp
}).catch(function () {});

```

```

        // синхронізуємо прогрес
        (пройдені рівні / розблокування скілів) у
        heroes/{id}.progress
        if (typeof SK.pushLocal ===
        'function') SK.pushLocal().catch(function
        () {});
    }
    });
}
} catch (e) {}
return s;
}

var _t = null;
function saveDebounce() {
    clearTimeout(_t);
    _t = setTimeout(save, 800);
}

/* — допомога тренажерам/квестам:
відмітити вивчений елемент —
Напр. у тренажері абетки:
SKProgress.markLearned('sk_train_ukrabc_1
earned_v1', 'A') */
function markLearned(key, item) {
    try {
        var a =
JSON.parse(localStorage.getItem(key) ||
'[]');
        if (!Array.isArray(a)) a = [];
        if (a.indexOf(item) === -1) {
            a.push(item); localStorage.setItem(key,

```

```
JSON.stringify(a)); }  
    } catch (e) {  
        try { localStorage.setItem(key,  
JSON.stringify([item])); } catch (e2) {}  
    }  
}  
  
window.SKProgress = {  
    UKR_TASKS: UKR_TASKS,  
    QUEST_XP_KEYS: QUEST_XP_KEYS,  
    ACCURACY_SKILLS: ACCURACY_SKILLS,  
    computeHealth: computeHealth,  
    computeAccuracy: computeAccuracy,  
    computeXP: computeXP,  
    computeUkrXP: computeUkrXP,  
    snapshot: snapshot,  
    save: save,  
    saveDebounce: saveDebounce,  
    markLearned: markLearned  
};  
})();
```